

Algoritmo(A)

{ $X = \emptyset$ ←

"ordina A in ordine decrescente" ← $O(n \log n)$

{ for $i = 1$ to $A.length - 1$ ← $O(n)$

{ for $j = i + 1$ to $A.length$ ←

{ $i = \text{Binary-Search}(A[j+1] \dots A.length,$
 $A[i] + A[j])$ ←

if $i > \emptyset$

then $X = X \cup \{A[i]\}$

$O(\log n)$

}

} return X ←

$O(n^2 \cdot \log n)$

$$\sum_{i=1}^{n-1} n - i$$

$O(n^2)$

13/1/2020

$P(m)$

{ sum = $\emptyset$

→ c

→ for $i = 1$ to $\log m$ → $O(\log m)$

{ $j = 2$ → c

→ while $j < 2^m$ → $O(m)$

{ sum = sum + $F(i)$ → $\Theta(\log i)$

→ $j = j * 2$

}

$\log m$

$\sum_{i=1}^{\log m} (m \cdot \log i) =$

$T(F(m)) = \Theta(\log m)$

$m \sum_{i=1}^{\log m} \log i$

$i = 1$

$Cw_1 \leftarrow$

$i = 2 +$

$Cw_2 \leftarrow$

$i = 3 +$

Cw_3

+

$\log m \cdot \log(\log m)$

$O(m \cdot \log m \cdot \log(\log m))$

+

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$\frac{n}{\log n}$$

CASO 1



$$\frac{n}{\log n} = O(n^{1-\epsilon})? \quad \Leftarrow$$

$$\frac{n}{\log n} \leq c \cdot \cancel{n^{1-\epsilon}}$$

$$\frac{\log n}{\cancel{n^{1-\epsilon}}}$$

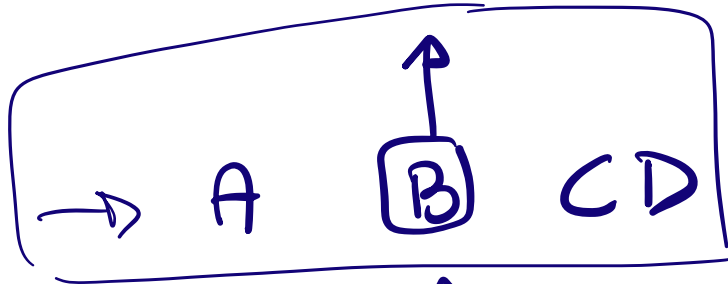
$$\frac{n^\epsilon}{\log n} \leq c$$

$$\frac{n^\epsilon}{\log n} \rightarrow \infty$$

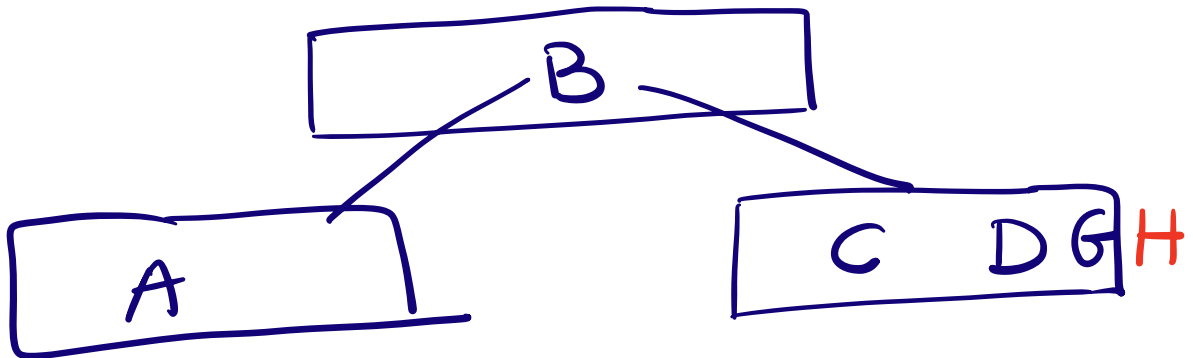
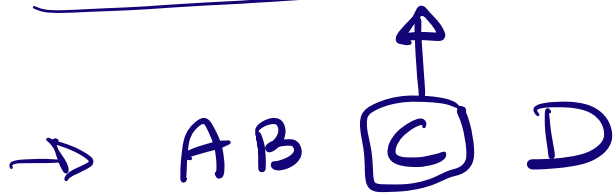
B-tree, $m=4$

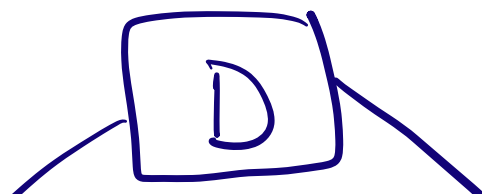
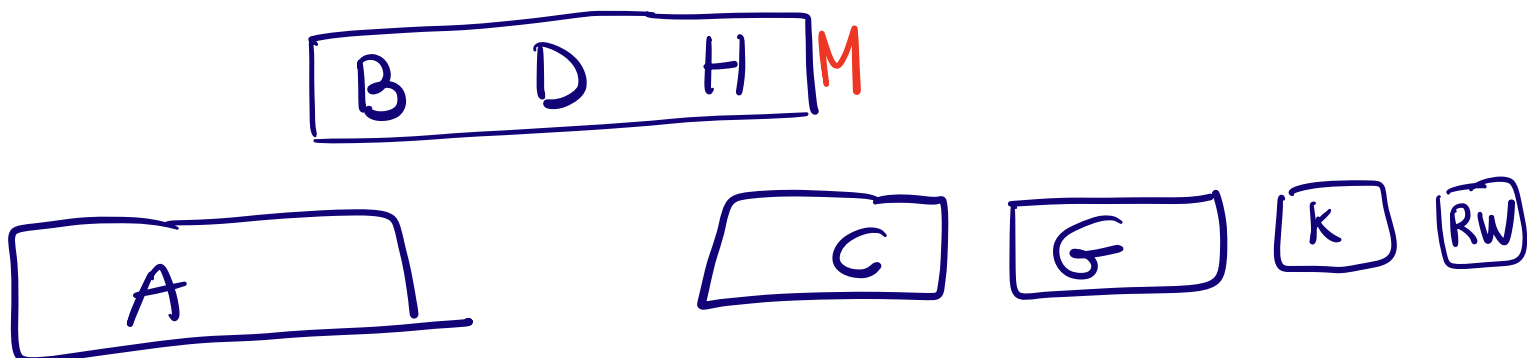
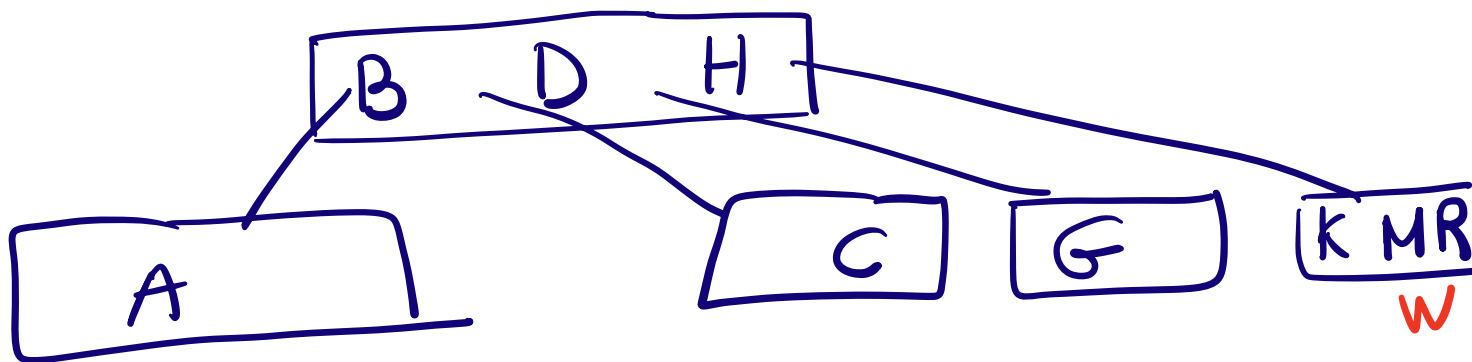
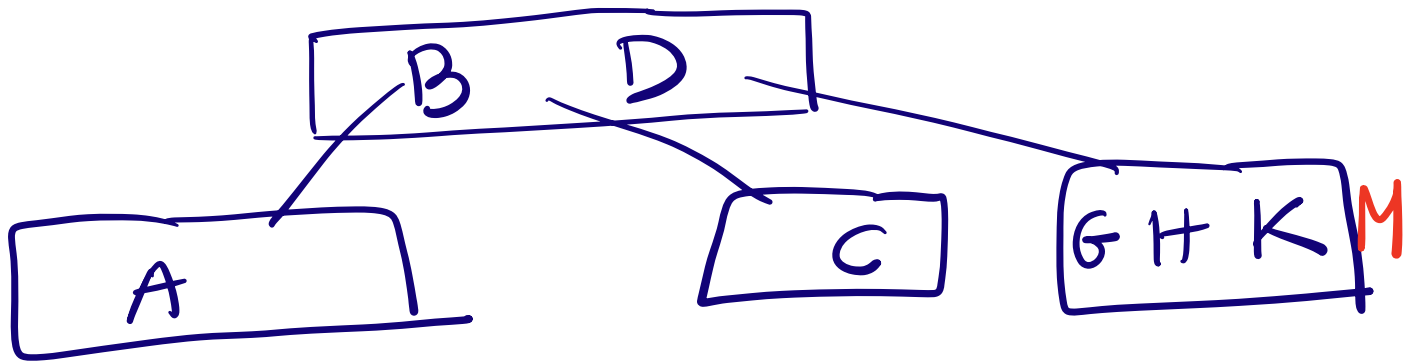
~~A, B, C, D, G, H, K, M, R, W, Z~~

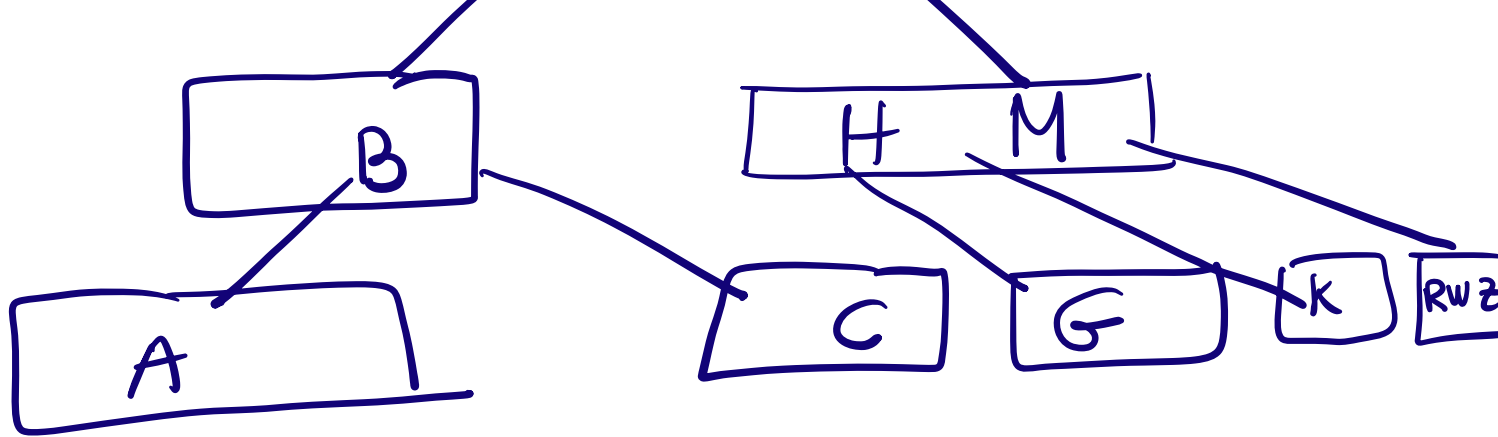
A B C D

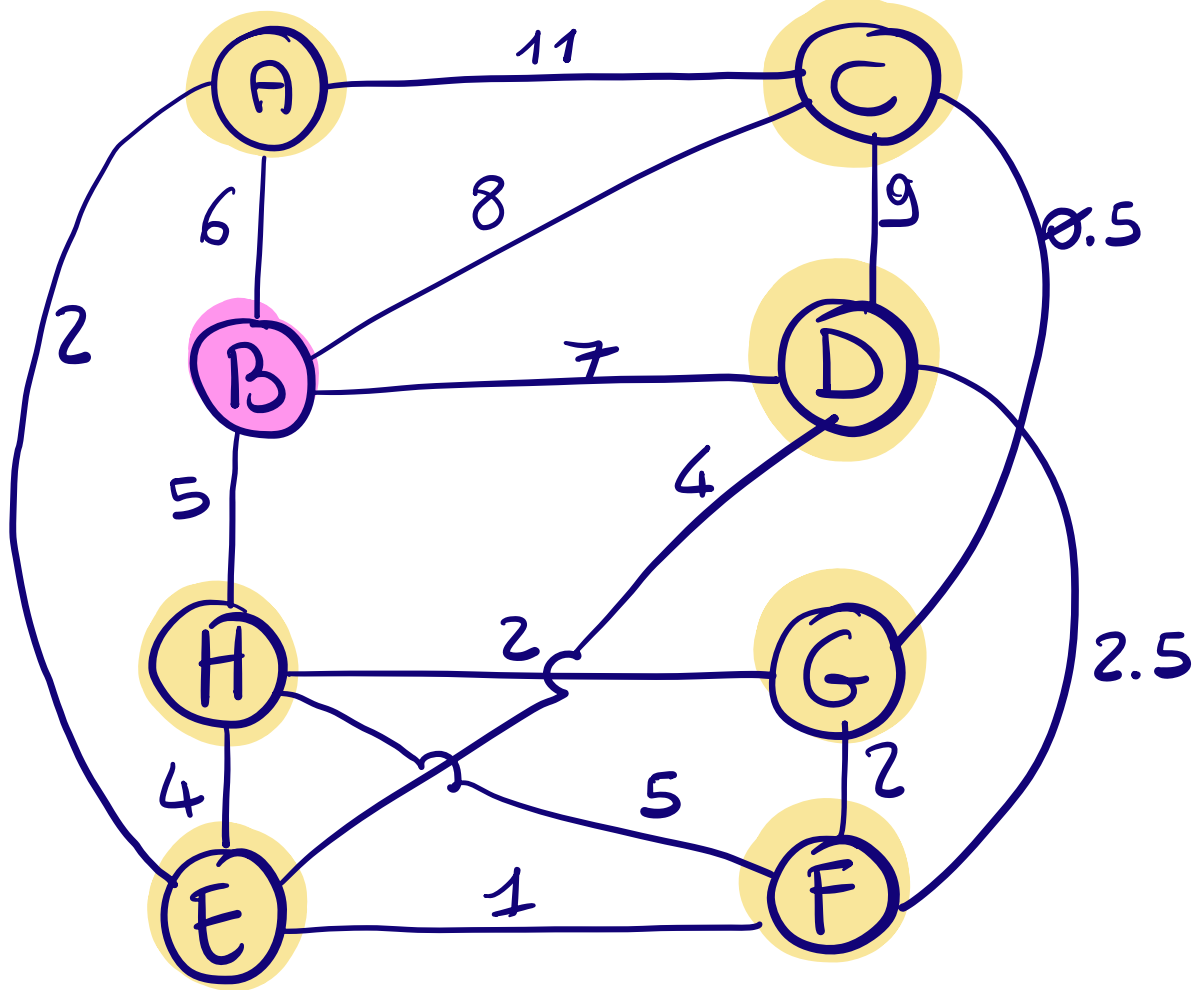


OPPURE

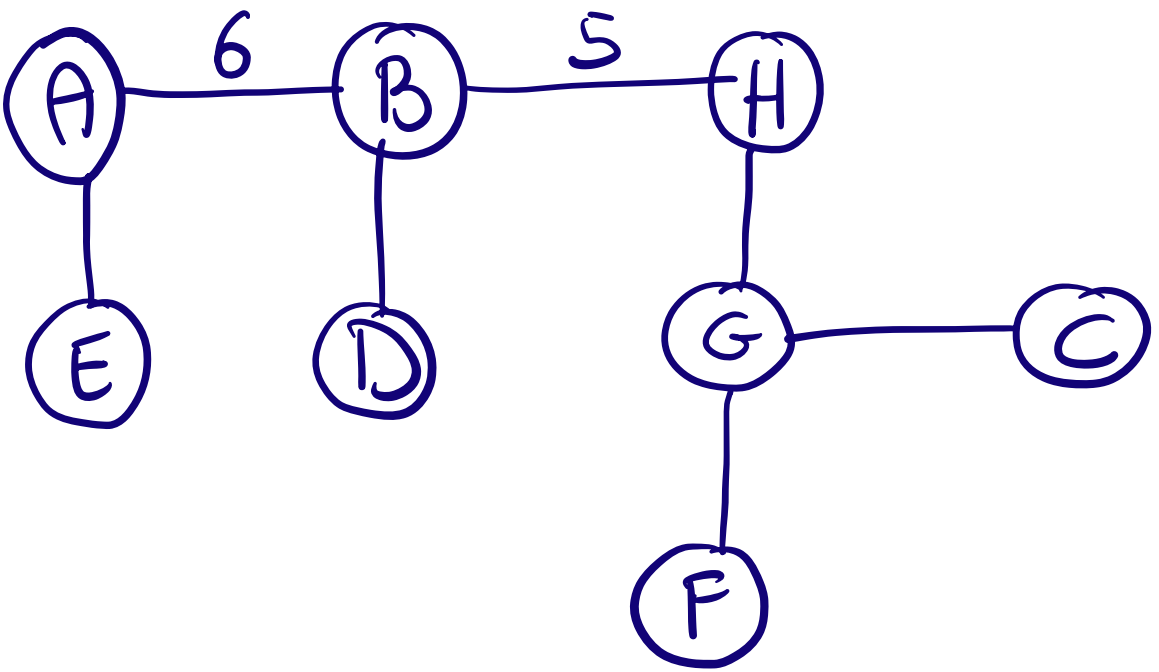








A	B	C	D	E	F	G	H
∞, N	∞, N	∞, N	∞, N	∞, N	∞, N	∞, N	∞, N
6, B		8, B	7, B	∞, N	∞, N	∞, N	5, B
6, B		8, B	7, B	9, H	10, H	7, H	
		8, B	7, B	8, A	10, H	7, H	
		8, B		8, A	9.5, D	7, H	
		7.5, G		8, A	9, G		
				8, A	9, G		
				9, G			



20/1/2021

$$T(n) = \underbrace{0.9 T(n/2)} + \frac{n}{\log n} \quad \leftarrow \begin{matrix} \text{NON SI} \\ \text{PUO'} \end{matrix}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n} \quad \leftarrow \begin{matrix} \text{NON SI} \\ \text{PUO'} \end{matrix}$$

Algoritmo (A)

{ $n = A.length$ —————> <

if $n == 1$ then return $A[\emptyset]$ —————> <

for $i = 0$ to $(n/2) - 1$ —————> $O(n)$

→ { $A1[i] = A[i]$ —————> <

→ { $A2[i] = A[n/2 + i]$ —————> <

for $i = 0$ to $(n/2) - 1$

for $j = i + 1$ to $(n/2) - 1$

if $(A1[i] == A2[j])$ then $A2[j] = \emptyset$ —————> <

$\sum_{i=0}^{n/2} \rightarrow O(n^2)$

```

    b1 = Algoritmo(A1) →
    b2 = Algoritmo(A2) →
    return max(b1, b2)
}

```

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n^2)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + \underline{n^2}$$

$$n^{\log_a b} = n^{\log_2 2} = \boxed{n} \quad \boxed{n^2}$$

CASO 3

$$\underline{\underline{n^2 = \Omega(n^{1+\epsilon}) \quad \epsilon < 1}}$$

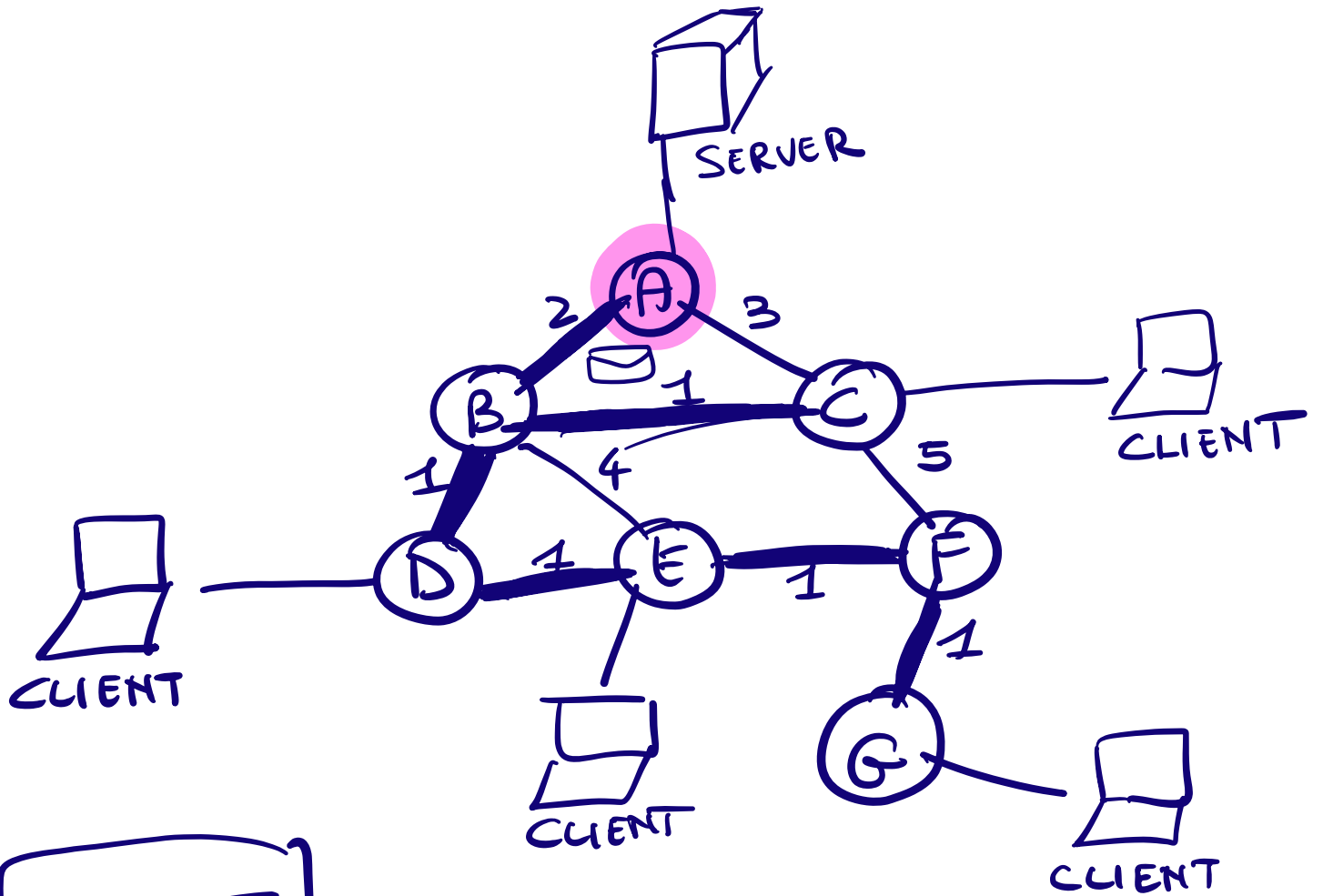
$$a f\left(\frac{n}{b}\right) \leq c \cdot f(n) \quad \leftarrow$$

$$2 \frac{\cancel{n^2}}{4} \leq c \cdot \cancel{n^2}$$

$$\frac{1}{2} \leq c$$

$$c = \frac{1}{2}$$

$$\Theta(f(n)) = \Theta(n^2)$$



M.S.T.